Neurolabs Cordova SDK Integration Guide

Technical Integration Specification

Document ID: NLB-CORDOVA-INTEGRATION-GUIDE
Version: v1.6.3
Date: 2026-07-20
Status: Partner integration reference
Owner: Neurolabs

1. Scope

This guide is for partner hybrid apps integrating ‘neurolabs-cordova-sdk‘.

2. Changes

The Cordova plugin is a thin bridge over the native SDKs, so most changes listed here come from the underlying ‘neurolabs-android-sdk‘ and ‘neurolabs-ios-sdk‘ releases that the plugin pulls via ‘scripts/prepare-sdk.js‘ at install time. No public JS API has been broken since v1.1.9.

v1.3.2

- Plugin pulls native SDK v1.3.2 binaries on ‘cordova plugin add‘:

Android AAR + iOS xcframework refreshed via the mobile-dist manifests.

- Native v1.3.2 highlights (no JS surface change required):

- Mixpanel ingestion now lands in the EU region (events were previously POSTed to the US endpoint and silently dropped). iOS also fixes a separate bug where Mixpanel rejected every event because ‘time‘ was serialized as a string rather than a JSON number. - Done button stays responsive while the SDK persists captures - disk I/O moved off the main thread on both platforms, inline spinner / progress indicator surfaced. - Android: native ‘SIGSEGV‘ fix in ‘com.google.ai.edge.litert.Model.nativeLoadAsset‘ when the TFLite asset is stored compressed in the APK; the loader probes the asset with ‘AssetManager.openFd‘ first and extracts to internal storage when the asset is not mmap-able. - Onboarding barcode pipeline normalises to GTIN-14 end-to-end and propagates the scanned barcode format through every product-lookup client. - iOS: new ‘NLOperationsCatalogClient‘ (in ‘ProductAuditKit‘) for the operations catalog; replaces ‘precondition‘-based validation with thrown errors so a malformed barcode no longer crashes the host app.

v1.2.5

- Native sequence-counter parity: Cordova bridge surfaces the

‘showsSequenceCounterLabel‘ (and ‘showsCapturedCountLabel‘) options to match the native SDK toggles.

- Demo SDK setup builds fixed; iOS Sentry injection timing hardened;

Android LiteRT dependency pinning correction; sequence-counter argument plumbed through correctly.

v1.2.3

- ‘openCamera‘ supports ‘cameraMode: "ar" | "default"‘ - ‘"default"‘

enables the 0.5/1 lens switcher on both iOS and Android.

- ‘init‘ supports ‘analyticsEnabled‘, ‘sentryEnabled‘, ‘sentryDsn‘ for controlling SDK analytics and crash reporting.
- Tightened default quality validation thresholds (blur, glare, perspective) across both platforms.

v1.2.2

- Internal: Android plugin integration handling refactor; no public JS API change.

v1.2.1

- Cordova install hooks hardened; demo integration updated.

v1.2.0

- Internal scaffolding for the v1.2.x line; no JS API change.

v1.1.9

- Init and per-session routing use ‘taskUUID‘.
- Native detector/model warmup is enforced in ‘init‘ and guarded in ‘openCamera‘.
- ‘openCamera‘ can return ‘MODEL_INIT_FAILED‘ when native model initialization fails.
- ‘autoCloseAfterCapture=true‘ now keeps preview enabled and closes after Save confirmation.
- Manual finish (‘Done‘) is available when ‘autoCloseAfterCapture=false‘ and ‘allowManualFinish=true‘.
- ‘cameraClosed‘ event now includes ‘message‘.
- ‘captureQueued‘ events are deferred while camera is open and flushed after ‘cameraClosed‘.

3. Install

Download the plugin package from ‘https://github.com/neurolaboratories/neurolabs-mobile-dist/releases‘. The install hooks download and wire up the native SDKs automatically - no manual AAR or xcframework placement needed.

Native baseline requirements:

- Android: ‘minSdkVersion 26‘, JDK 17, Kotlin 2.x.
- iOS: deployment target iOS 17.0+, Xcode 16.4+ recommended (release toolchain baseline).

Android (Windows or macOS)

```
# Add the Android platform first, then install the plugin
cordova platform add android
cordova plugin add /path/to/neurolabs-cordova-sdk-vX.Y.Z.tgz
```

The ‘before_plugin_install‘ hook downloads ‘neurolabs-android-sdk.aar‘ from the matching GitHub release, stores it inside the plugin, and ‘plugin.xml‘ copies it to ‘app/libs/‘ automatically. ‘neurolabs.gradle‘ wires it into the build via ‘flatDir‘.

iOS (macOS only)

```
# Add the iOS platform first, then install the plugin
cordova platform add ios
cordova plugin add /path/to/neurolabs-cordova-sdk-vX.Y.Z.tgz
```

The hook downloads ‘NeurolabsSDK.xcframework‘ from the matching GitHub release and Cordova embeds it automatically during build. Requires ‘curl‘ (standard on macOS) or ‘gh‘ CLI. For private releases set ‘GITHUB_TOKEN‘.

The iOS install hook injects the ‘Sentry‘ Swift Package dependency into the generated Xcode project automatically during platform preparation. If Xcode still reports ‘unable to find module dependency: Sentry‘, re-run ‘cordova prepare ios‘ so the package graph refreshes in the generated project.

Optional overrides (use a pre-downloaded file or a specific URL instead of auto-download):

```
# Local file
NEUROLABS_IOS_XCFRAMEWORK_ZIP=/path/to/NeurolabsSDK.xcframework-vX.Y.Z.zip \
  cordova plugin add /path/to/neurolabs-cordova-sdk-vX.Y.Z.tgz

NEUROLABS_ANDROID_AAR_PATH=/path/to/neurolabs-android-sdk-vX.Y.Z.aar \
  cordova plugin add /path/to/neurolabs-cordova-sdk-vX.Y.Z.tgz

# Direct URL
NEUROLABS_IOS_XCFRAMEWORK_URL=https://github.com/.../NeurolabsSDK.xcframework-vX.Y.Z.zip \
  cordova plugin add /path/to/neurolabs-cordova-sdk-vX.Y.Z.tgz

NEUROLABS_ANDROID_AAR_URL=https://github.com/.../neurolabs-android-sdk-vX.Y.Z.aar \
  cordova plugin add /path/to/neurolabs-cordova-sdk-vX.Y.Z.tgz
```

iOS + Android (macOS)

Add both platforms before installing the plugin - both hooks run in a single ‘cordova plugin add‘.

4. SDK Initialization + Warmup

```
const Neurolabs = cordova.require('ai.neurolabs.cordova.Neurolabs');

await Neurolabs.init({
  apiKey: '<API_KEY>',
  operationsApiKey: '<OPERATIONS_API_KEY>', // REQUIRED for uploads (see note below)
  // operationsBaseUrl: 'https://api.operations.staging.neurolabs.ai/v1', // optional overrid
e (staging / self-hosted)
  apiBaseUrl: 'https://api.neurolabs.ai/v2',
  taskUUID: '<DEFAULT_TASK_UUID>',
  allowBase64PhotoExport: false,
  analyticsEnabled: true,          // SDK analytics (default true)
  sentryEnabled: true,             // Sentry crash reporting (default true)
  // sentryDsn: 'https://...',     // optional custom Sentry DSN
  // yoloModelVariant: 'shelf_rows', // see below
});
```

> ** Upgrade note (pre-1.6 integrators): ‘operationsApiKey‘ is now required for uploads.** > > The legacy ‘/images‘ upload endpoint has been removed - the native > capture queue is operations-only. If you call ‘init()‘ **without** > ‘operationsApiKey‘, ‘init()‘ and ‘openCamera()‘ still succeed and photos > still queue, but **every upload fails silently**: the queue item errors > with ‘OPERATIONS_KEY_REQUIRED‘ (non-retryable), surfaced only through the > ‘uploadFailed‘ event. There is no console error and no init-time > rejection, so this is easy to miss during an upgrade. > > ‘‘‘js > Neurolabs.addListener(’uploadFailed’, ({ errorCode, message }) => { > if (errorCode === ’OPERATIONS_KEY_REQUIRED’) { > console.error(’Missing operationsApiKey in init() - uploads will never succeed.’, message); > } > }); > ‘‘‘ > > ‘operationsBaseUrl‘ (optional) overrides the operations endpoint for > staging / self-hosted deployments. It must be an ‘https‘ URL with a host > and no embedded credentials; ‘http‘, malformed, or userinfo-bearing > values reject at ‘init()‘ with ‘INVALID_ARGUMENT‘. Omit it to keep the > production default (‘https://api.operations.neurolabs.ai/v1‘).

YOLO model variant

‘yoloModelVariant‘ (optional, default ‘nlb_model‘) picks the detector model the SDK loads. Accepted values, case-insensitive:

Value	Description
‘nlb_model‘	Legacy product detector (default).		‘yolo_v26‘	YOLOv2.6 product detector.		‘yolo_v11_seg‘	YOLO11-Seg product detector with mask coefficients.		‘shelf_rows‘	May-26 shelf-row detector - emits per-row regions + masks. Drives the row-aware live guidance (per-row quality chips, mask-based tilt/yaw, lock-on overlay). Requires the ‘weights_may_26.tflite‘ / ‘weights_may_26_labels.json‘ assets to be bundled.

Passing an unknown value raises ‘INVALID_ARGUMENT‘ rather than silently falling back to ‘nlb_model‘.

Warmup is native-side; monitor progress through ‘loadingProgress‘ event.

```
Neurolabs.addListener(’loadingProgress’, (payload) => {
  console.log(’loading’, payload);
});
```

5. Queue Management

```
await Neurolabs.setAutoSyncEnabled(true);
await Neurolabs.setWifiOnlyUploadsEnabled(false);
await Neurolabs.setDeletePhotosOnUploadSuccess(true);

await Neurolabs.pauseQueue();
await Neurolabs.resumeQueue();
await Neurolabs.setRetryPolicy({ retryCount: 5 });

const status = await Neurolabs.getQueueStatus();
console.log(’queue status’, status);

await Neurolabs.flushQueue();
await Neurolabs.retryFailedUploads();
```

6. Open Custom Camera

```
await Neurolabs.openCamera({
  sessionId: crypto.randomUUID(),
  type: 'shelf',
  cameraMode: 'default',          // 'ar' or 'default' - enables 0.5/1 lens switcher
  guidanceMode: 'strict',
  validationPreset: 'ios_parity',

  confidenceThreshold: 0.25,
  iouThreshold: 0.45,
  maxCaptures: 1,

  showAlignmentGuidance: true,
  enableValidation: true,
  liveQualityChecksEnabled: true,
  liveQualityTargetFps: 6,

  // keep custom-camera guidance UI path
  showDetections: false,
  showCapturedRegions: false,

  // optional metadata/cropping
  sendDetectionsMetadata: true,
  enablePreviewCropping: true,

  // now defaults to true if omitted, but explicit is clearer
  autoCloseAfterCapture: true,

  // manual-finish mode (Done button):
  // allowManualFinish: true,
  // minCapturesBeforeDone: 3,

  // per-session routing override
  taskUUID: 'bfc85982-b955-4f65-9f32-b6dbed85f364',

  // image size constraints
  maxImageDimension: 1920,
  maxImageWidth: 1920,
  maxImageHeight: 1920,
  imageCompressionQuality: 0.85
});
```

Manual-finish example:

```
await Neurolabs.openCamera({
  sessionId: crypto.randomUUID(),
  type: 'shelf',
  guidanceMode: 'strict',
  maxCaptures: 10,
  autoCloseAfterCapture: false,
  allowManualFinish: true,
  minCapturesBeforeDone: 3
});
```

6a. Multi-Bay Capture (wide shelves)

Multi-bay capture (native SDK v1.6.0+) guides the user across several overlapping "bays" of a wide shelf in a single session and de-duplicates detections across them. Enable it by passing a ‘multiBay‘ options object to ‘openCamera‘:

```
await Neurolabs.openCamera({
  sessionId: crypto.randomUUID(),
  type: 'shelf',
  guidanceMode: 'strict',
  multiBay: {
    minBays: 2,                  // 1..8
    maxBays: 4,                  // 2..8
    targetOverlapFraction: 0.30, // 0.05..0.9 (native default 0.30)
    onDeviceDedup: true,         // de-duplicate detections across bays on device
    generatePreview: true,       // build a stitched display-only preview
    previewMaxDimension: 2048    // max long-edge px of the stitched preview
  }
});
```

Out-of-range values reject with ‘INVALID_ARGUMENT‘. An optional ‘guidanceStyle: ’classic’ | ’glance’‘ (case-insensitive) selects the guidance overlay style.

Attach free-text submit notes to the session before its payload uploads:

```
await Neurolabs.setMultiBaySubmitNotes('Aisle 4, promo end-cap', sessionId);
```

When a multi-bay session completes, the ‘cameraClosed‘ payload carries an extra ‘multiBay‘ summary (forwarded untouched from the native SDK). The per-photo ‘captureQueued‘ / upload events are unchanged:

```
Neurolabs.addListener('cameraClosed', ({ sessionId, cancelled, captureCount, message, multiBa
y }) => {
  if (!multiBay) return; // single-bay session
  const {
    countsByLabel,        // { [label]: count } de-duplicated across bays
    baysCount,            // number of bays captured
    dedupTrusted,         // false if the cross-bay alignment chain broke
    previewImageFileUri   // 'file://...' when generatePreview: true (optional)
  } = multiBay;
  console.log('multi-bay summary', baysCount, dedupTrusted, countsByLabel);
});
```

‘dedupTrusted: false‘ means the on-device de-duplication could not be fully trusted (e.g. insufficient overlap) - the backend re-de-duplicates authoritatively, so treat ‘countsByLabel‘ as provisional in that case.

7. Post-Processing + Lifecycle Events

```
// Fires when the camera screen is dismissed (Done or Close pressed).
// captureCount is the number of photos queued for upload - NOT the photo data itself.
// To retrieve photo data, use getPhoto() with the captureId from captureQueued (see section
8).
Neurolabs.addListener('cameraClosed', ({ sessionId, cancelled, captureCount, message }) => {
  console.log('cameraClosed', sessionId, cancelled, captureCount, message);
});

// Fires once per photo taken, immediately after each capture is added to the upload queue.
// captureQueued events are buffered while the camera is open and always delivered after came
raClosed.
// Use the captureId here to retrieve the local photo via getPhoto().
Neurolabs.addListener('captureQueued', ({ captureId, queueItemId, sessionId }) => {
  console.log('queued', captureId, queueItemId);
});

// Fires after the Neurolabs server has processed an uploaded photo and returned an analysis
result.
// This is a server-side callback - it does NOT fire when the photo is taken locally.
// Payload: { captureId, sessionId, success, message, detectionCount, capturedRegionCount }
Neurolabs.addListener('captureResult', (payload) => {
  console.log('captureResult', payload);
});

Neurolabs.addListener('uploadSucceeded', (item) => console.log('uploaded', item));
Neurolabs.addListener('uploadFailed', (payload) => console.warn('uploadFailed', payload));
Neurolabs.addListener('queueStatusChanged', (status) => console.log('queueStatusChanged', sta
tus));
```

8. Retrieving Photos Locally

Photos are stored in the device's app cache after capture. Use 'getPhoto()' to access them by 'captureId' (from 'captureQueued') or 'queueItemId'.

```
const capturedIds = [];

Neurolabs.addListener('captureQueued', ({ captureId }) => {
  capturedIds.push(captureId);
});

Neurolabs.addListener('cameraClosed', async ({ cancelled }) => {
  if (cancelled) return;
  for (const captureId of capturedIds) {
    // format: 'fileUri' returns { uri: 'file://...' } - a temp path in the app cache
    // format: 'base64'  returns { base64: '...' }    - requires allowBase64PhotoExport: true
 in init()
    const photo = await Neurolabs.getPhoto({ captureId, format: 'fileUri' });
    console.log('photo uri', photo.uri);
  }
  capturedIds.length = 0;
});
```

'deletePhoto()' accepts the same query shape ('captureId', 'queueItemId', or 'responseId') and removes the local file from the cache.

9. Notes

- 'openCamera' is the custom native camera entrypoint.

- Use the strict shelf payload above for full shelf guidance checks and auto-close after a validated save.
- ‘liveQualityChecksEnabled=true‘ is required for pill/rotation/warning/error guidance behavior.
- Keep ‘type: ’shelf’‘, ‘guidanceMode: ’strict’‘, ‘showDetections: false‘, and ‘showCapturedRegions: false‘ for the custom-camera guidance UI path.
- ‘guidanceMode: ’guidance’‘ should only be used as a temporary fallback if a partner explicitly wants looser Android behavior than the parity recommendation.
- ‘autoCloseAfterCapture=true‘ closes after Save from preview/review flow.
- ‘Done‘ appears only in manual-finish mode: ‘autoCloseAfterCapture=false‘ + ‘allowManualFinish=true‘.
- Use ‘minCapturesBeforeDone‘ to require a minimum number of captures before ‘Done‘ becomes active.
- If ‘allowManualFinish=false‘, ‘maxCaptures‘ acts as a hard limit and capture disables at the limit.
- ‘captureQueued‘ events are buffered while the camera is open and always delivered after ‘cameraClosed‘ - never during an active session.
- Keep base64 transport disabled unless explicitly needed.

Troubleshooting: "Unresolved reference" Kotlin errors on Android build

Symptoms like ‘Unresolved reference ’setOperationsBaseUrl’‘, ‘’setMultiBaySubmitNotes’‘, ‘’resourceName’‘, or ‘’labelsResourceName’‘ during the Android build mean the plugin source is being compiled against a **stale ‘neurolabs-android-sdk.aar‘** - an old AAR left in ‘platforms/android/app/libs/‘ (which takes precedence) from a previous plugin version.

Fix:

1. Delete the stale AAR and its metadata: ‘platforms/android/app/libs/neurolabs-android-sdk.aar‘ (+ ‘.meta.json‘) 2. Re-run ‘cordova prepare android‘ so the plugin re-fetches the AAR matching the plugin’s pinned native version. 3. Rebuild.

Since v1.6.3 the Android Gradle script fails the build early with an explicit version-mismatch message (instead of the confusing Kotlin errors) whenever the resolved AAR’s version does not match the plugin’s ‘neurolabs.sdkVersions.android‘.